3D Euler: the first optimization experiment

A staged investigation of optimal solvers within a restricted class.

Current result: caching old-state thermodynamics preserved every tested update and reduced one-step CPU time. **No 3D Euler solver optimality theorem has been proved.**

The first gate

The experiment asks a deliberately bounded question: can an AI-assisted workflow improve the implementation of a fixed three-dimensional fluid discretization, preserve its output, and retain enough evidence to support a later proof? It does not yet search over numerical methods or establish the fastest route to a given PDE accuracy.

Item	Fixed contract
Equations	3D compressible Euler; ideal gas; gamma = 2; density, three momenta, total energy
Discretization	Uniform periodic Cartesian grid; first-order shared-face Rusanov flux; forward Euler
Candidate change	Compute pressure and sound speed once from the old state; reuse them for x, y and z fluxes
Output requirement	Same complete discrete update for the same state and supplied timestep
Measured objective	CPU time per full update kernel, including cache and output allocation

Why this is a first step

The previous 1D Poisson baseline had a direct linear solve and an output-size lower bound. Those facts do not transfer to a full nonlinear Euler evolution. The present candidate changes implementation work only. A broader claim will need its own algorithm class, accuracy contract, and lower bound.

Measured evidence

One warm-up call per kernel; 11 timing trials per grid; deterministically shuffled execution order. Times include cache creation and returned-array allocation. Timestep selection and validation are outside the timing interval.

Grid	Reference (ms)	Cached (ms)	Ratio
16^3	0.800	0.686	1.166×
32^3	8.699	8.317	1.046×
48^3	25.073	20.848	1.203×
64^3	52.537	42.064	1.249×

Update and admissibility checks

Input	Grid / steps	Equality	Minimum rho / p
random	8^3 / 8	Bitwise identical	0.8147 / 1.0602
random	12^3 / 8	Bitwise identical	0.7582 / 0.9455
coupled	16^3 / 20	Bitwise identical	0.9190 / 0.9441
blast	24^3 / 20	Bitwise identical	0.8734 / 0.1000

All checked steps retained positive density and pressure. The largest absolute drift of a volume-weighted conserved total was 1.66e-14. Pressure and sound-speed evaluations decrease from 3N to N by source inspection; this count has not been proved minimal.

What these measurements do not establish

The random and coupled cases exercise nonlinear three-dimensional states. The spherical pressure blast is an execution/admissibility check without an independently validated shock solution. There is no convergence result for these cases. Bitwise equality on finite tests is not an all-state formal equivalence theorem.

The timing ratio compares two NumPy implementations on one CPU host. It does not prove a best constant, end-to-end speedup, GPU advantage, or optimal time to a prescribed error. The roughly 1.05× result at 32^3 is small enough to warrant repetition under controlled host load.

Linux-6.18.54.1-microsoft-standard-WSL2-x86_64-with-glibc2.39

Host: AMD Ryzen 9 9900X. Python 3.12.13; NumPy 2.5.1. This experiment uses CPU execution.

Proof gates and replay

Gate	Required evidence	Status
1. Preserve the update	Frozen source, nonlinear 3D trajectories, conservation/admissibility checks, all timing trials	Empirically passed
2. Formalize implementation	Actual Euler flux; admissibility assumptions; exact-arithmetic cache equivalence; allowed dataflow transformations; meaningful cost lower bound	Next
3. Solve to an error target	Manufactured smooth solutions, independent shock references, refinement studies, all timesteps and setup costs	Open
4. Discover broader methods	New flux/reconstruction/time-integrator candidates, certified counterexamples, matched-accuracy baselines, new lower bounds	Open-ended

The next theorem must be named at its actual scope. A minimum count of thermodynamic evaluations would establish that subproblem's optimum, not optimality of an entire fluid solver. If only a linear output-size bound is available, the report must say so.

Reproduce locally

From the repository root:

```
.venv/bin/python demos/lean_euler/optimization/run_local.py
--output artifacts/lean-euler/optimization-replay-001
```

Choose a new output directory. The driver submits an exploratory Simjecture lab job, freezes experiment.py and euler.py, and retains the receipt and optimization.json. The committed record stores all samples and the source SHA256 hashes. The study has no independent scientific review.

Sources and provenance

The candidate and measured evidence are in PR #90: github.com/tomzhu0225/simjecture/pull/90. The PR is draft while the 3D research develops.

[1] X. Zhang and C.-W. Shu (2010), positivity-preserving Euler schemes on rectangular meshes. This provides established context for separate admissibility arguments, not an optimality proof for this candidate. Author-hosted paper.

[2] Clawpack, Riemann solvers: directional interface solvers in multidimensional methods. Official documentation.

experiment.py SHA256:
b70d8a26ca230938e1bd8c14ee2ff478472cf19fe8609825b73a76bac9833856

euler.py SHA256:
505ebff19d161561f3723a259e65e9476e7c0b40c357146df905dfa56e95d2c2